A

Mini Project Report

on

UniAccess: Network Analyzer

Submitted in partial fulfillment of the requirements for the

degree

Second Year Engineering – Computer Science and Engineering (Data Science)

by

Kunal Thakur 24107027

Parth Mane 24107127

Pratham Shetty 24107088

Tanmay Monde 24107034

Under the guidance of

Ms. Rajashri Chaudhari

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W)-400615

UNIVERSITY OF MUMBAI

Academic year: 2025-26

CERTIFICATE

This to certify that the Mini Project report on UniAccess: Network Analyzer has been submitted by Tanmay Monde (24107034), Kunal Thakur (24107027), Parth Mane (24107127) and Pratham Shetty (24107088) who are bonafide students of A. P. Shah Institute of Technology, Thane as a partial fulfillment of the requirement for the degree in **Computer Science and Engineering (Data Science)**, during the academic year **2025-2026** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Ms. Rajashri Chaudhari
Guide

Dr. Pravin Adivarekar
HOD, CSE(Data Science)

Dr. Uttam D. Kolekar
Principal

External Examiner:
1.

Internal Examiner:
1.

Place: A. P. Shah Institute of Technology, Thane
Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide **Ms. Rajashri Chaudhari** Expressing gratitude towards our HOD, **Dr. Pravin Adivarekar**, and the Department of Computer Science and Engineering (Data Science) for providing us with the opportunity as well as the support required to pursue this project. We would also like to thank our project coordinator **Ms. Richa Singh and Mr. Kiran Gaikwad** who gave us her valuable suggestions and ideas when we were in need of them. We would also like to thank our peers for their helpful suggestions.

TABLE OF CONTENTS

1. Introduction……..1

   1.1.Purpose.....1

   1.2.Problem Statement……...2

   1.3.Objectives..... 3

   1.4.Scope.....…...4

2. Proposed System…….....5

   2.1.Features and Functionality……...5

3. Project Outcomes.....…...9

4. Software Requirements.....11

5. Project Design……...…….....14

6. Project Scheduling.....18

7. Results…….19

8. Conclusion…….....23

References

Chapter 1

Introduction

UniAccess is a university-level Network Access Control (NAC) and real-time traffic analysis system designed to enhance the security and operational management of institutional campus networks. Modern academic institutions face a growing challenge: hundreds of heterogeneous devices including student laptops, smartphones, IoT sensors, and departmental servers share a common network infrastructure, creating a broad attack surface that traditional static security tools cannot adequately protect. UniAccess addresses this gap by providing automated device registration, real-time packet-level monitoring, anomaly-based flood detection, and a mobile-first administrator interface, all integrated into a single deployable system.

The system is built on a modern asynchronous technology stack. The administrator interacts with a Flutter-based mobile application that communicates with a Python FastAPI backend over HTTPS REST APIs and persistent WebSocket connections. The backend performs live packet capture using TShark, the command-line interface of Wireshark, and stores all device records, events, and policies in a PostgreSQL database accessed via the asyncpg asynchronous driver. This chapter introduces the problem context, the purpose of this document, the specific objectives of the project, and the boundaries of its scope.

1.1 Purpose

UniAccess is a university-level Network Access Control and real-time traffic analysis system. The core purpose of this system is to address the security and operational management challenges faced by academic campus networks, where hundreds of heterogeneous devices including student laptops, smartphones, IoT sensors, and departmental servers share a common network infrastructure.

The system is designed to provide network administrators with automated device registration, real-time packet-level monitoring, and anomaly-based flood detection through a mobile-first administrator interface. It specifically targets the detection and automated blocking of internal flood-based attacks — TCP SYN floods, UDP floods, and ICMP floods

which are particularly dangerous in campus environments because the attacking device has already bypassed the perimeter firewall.

The backend is built on Python FastAPI communicating over HTTPS REST APIs and persistent WebSocket connections, performing live packet capture using TShark and storing all data in a PostgreSQL database. The administrator interacts with the system through a Flutter-based mobile application, allowing them to monitor, manage, and respond to network threats from anywhere without needing to be physically present at a network console.

1.2 Problem Statement

The management of network access in academic institutions presents a set of challenges that are distinct from enterprise environments. Universities operate open-access policies where students, faculty, and visitors connect personal and institutional devices simultaneously. Network administrators in such environments lack real-time visibility into which devices are currently connected, what traffic patterns they are generating, and whether any device is behaving anomalously.

Existing tools such as static firewall rules and manual MAC address whitelists are configured infrequently and cannot respond to dynamic threats. When a student device becomes compromised or a misconfigured application begins generating excessive traffic, the current approach requires the administrator to be physically present at a network console to investigate and intervene. This delay can allow a flood-based denial-of-service attack to disrupt the entire campus network, affecting examinations, online learning platforms, and administrative systems.

Furthermore, flood-based attacks, including TCP SYN floods, UDP floods, and ICMP floods, are particularly damaging in campus environments because the attacker is typically an internal device that has already bypassed the perimeter firewall. Identifying the source MAC address of such an attack and blocking it in near-real time requires packet-level capture and automated policy enforcement that existing campus tools do not provide.

The absence of a mobile-accessible administration interface further compounds the problem. A network administrator who is not seated at their workstation has no means of viewing the current device list, checking packet rates, or issuing a block command. There is a clear and immediate need for a system that combines real-time packet capture, threshold-

based anomaly detection, automated enforcement, and a mobile admin interface into a single lightweight deployable tool.

1.3 Objectives

The primary objectives of the UniAccess project are:

- To design and implement a Network Access Control system that registers, authorizes, and monitors device MAC addresses on the campus network, providing a persistent and queryable device registry backed by a relational database.
- To integrate TShark as a managed subprocess within the FastAPI backend to perform live packet capture on the campus network interface, parsing packet metadata including source MAC, destination IP, protocol type, and packet size in real time.
- To implement automated detection of network flood attacks, specifically TCP SYN floods, UDP floods, and ICMP floods, using configurable per-device per-protocol packet-rate thresholds evaluated at one-second sampling intervals.
- To develop a cross-platform Flutter admin mobile application providing a device list dashboard, device detail view, live packet stream, event log, and alert notification system accessible from any Android device.
- To implement JWT-based stateless authentication using python-jose and passlib with bcrypt, ensuring all API endpoints are protected and accessible only to authenticated administrators.
- To provide real-time event and alert streaming from the backend to the admin application using a persistent WebSocket connection, with end-to-end alert latency under 500 milliseconds.
- To persist all device registrations, traffic log entries, block events, session tokens, and policy configurations in a normalized PostgreSQL database with referential integrity and concurrent access support via asyncpg connection pooling.

1.4 Scope

The scope of the UniAccess system covers the following functional areas within the boundaries of this mini project:

In scope: Device management including MAC address registration, activation, blocking, and unblocking with owner attribution; real-time packet capture on a specified network interface using TShark with JSON output parsing; automated SYN, UDP, and ICMP flood detection with auto-block and event logging; a Flutter admin mobile application for Android with JWT-secured login, device list, device detail, live WebSocket packet stream, and event log screens; a Python FastAPI backend with asynchronous REST endpoints and WebSocket server running on Uvicorn ASGI; a PostgreSQL database schema with five normalized tables and asyncpg-based connection pooling; configurable flood detection policies stored in the database and applied at runtime.

Out of scope for this version: Distributed deployment across multiple campus buildings or subnets; deep application-layer packet inspection beyond layer 3/4 headers; integration with third-party RADIUS, LDAP, or 802.1X AAA servers; iOS support for the Flutter application; automated scheduled report generation; and machine learning-based anomaly detection beyond threshold rules. These represent natural directions for future enhancement and are noted in the Conclusion chapter.

Chapter 2

Proposed System

The proposed UniAccess system follows a three-tier client-server architecture where the Flutter admin mobile application communicates with the FastAPI backend over HTTPS REST APIs and WebSocket connections, and the backend interacts with the PostgreSQL database for all persistent operations while invoking TShark as a subprocess for live packet capture.

2.1 Features and Functionality

UniAccess provides the following features for the network administrator. Each feature is described with its mechanism and the specific components involved.

Admin Login and JWT Authentication:

The administrator logs in through the Flutter application by submitting a username and password to the POST /auth/login endpoint. FastAPI uses passlib with bcrypt to verify the password hash stored in the admin_users table. Upon successful verification, a signed JSON Web Token (JWT) is generated using python-jose with an HS256 algorithm and a configurable expiry period. The token is returned to the Flutter app and stored using Flutter Secure Storage. All subsequent API requests include this token in the Authorization: Bearer header. The backend validates the token on every protected endpoint; expired or tampered tokens return HTTP 401, redirecting the app to the login screen.

Device Registration and Management:

The admin can register any network device by providing its MAC address, a human-readable device name, and owner details. The registration endpoint performs a uniqueness check on the MAC address before inserting a new record into the devices table. Registered devices are assigned an Active status by default. The admin can deactivate a device, which removes it from live monitoring without deleting its history, or delete it entirely. The device list screen in the Flutter app displays all registered devices with their MAC address, IP address, current status badge (Active / Blocked), and real-time packet rate, updated via periodic REST polling.

Live Packet Capture and Stream:

The Capture Module spawns TShark as a managed asyncio subprocess targeting the configured network interface in monitor mode. TShark outputs captured packets as a continuous JSON stream, each record containing the source MAC address, destination IP, protocol (TCP, UDP, ICMP), packet length, and capture timestamp. The Capture Module reads this stream asynchronously, parses each JSON record, increments the per-device packet counter for the appropriate protocol, and forwards the record to the WebSocket broadcast queue. The Live Packet Stream screen in the Flutter admin app maintains a persistent WebSocket connection and renders each incoming packet entry in a scrollable log, with flood-source entries highlighted in red. The stream updates in under 500 milliseconds from capture to display.

Flood Attack Detection and Auto-Block Engine:

Three protocol-specific counters are maintained per device: TCP/SYN packet count, UDP packet count, and ICMP packet count. Each counter is compared against the corresponding threshold value retrieved from the active policy record in the policies table. If any counter exceeds its threshold, the engine immediately executes the auto-block procedure: the device record is updated to Blocked status in the devices table, a new entry is written to the blocked_ips table with the reason, timestamp, and trigger source marked as auto, and a structured alert message is pushed to all active WebSocket sessions. A notification banner appears on the admin application regardless of which screen is currently open.

Manual Block and Unblock:

In addition to automatic blocking, the administrator can manually block or unblock any registered device directly from the device detail screen. The Flutter app presents a Block button (styled in red) and an Unblock button (styled in green) at the bottom of the device detail view. Tapping either button triggers a confirmation dialog to prevent accidental operations. Upon confirmation, the app sends an authenticated PUT request to the /devices/{id}/block or /devices/{id}/unblock endpoint. The backend validates the JWT token, updates the device status in the devices table, writes an event record to the blocked_ips table with the trigger source set to manual and the admin username, and returns the updated device object. The change is reflected immediately on the device detail screen and propagated to the device list on the next refresh

Event Log:

The event log maintains a complete, timestamped audit trail of all significant system events. Every device registration, status change (block or unblock), flood detection alert, and admin login is recorded in the database. The Event Log screen in the Flutter app fetches entries from the GET /events endpoint, which supports pagination and optional filtering by device ID, event type, and date range. Each log entry displays the event type label, the MAC address of the affected device, the trigger source (auto or admin), and the ISO 8601 timestamp. The event log is read-only within the app and provides administrators with full historical context for any device on the network.

2.2 Block Diagram

The block diagram below illustrates the high-level architecture of the UniAccess system, showing the three main tiers and the flow of data between components.

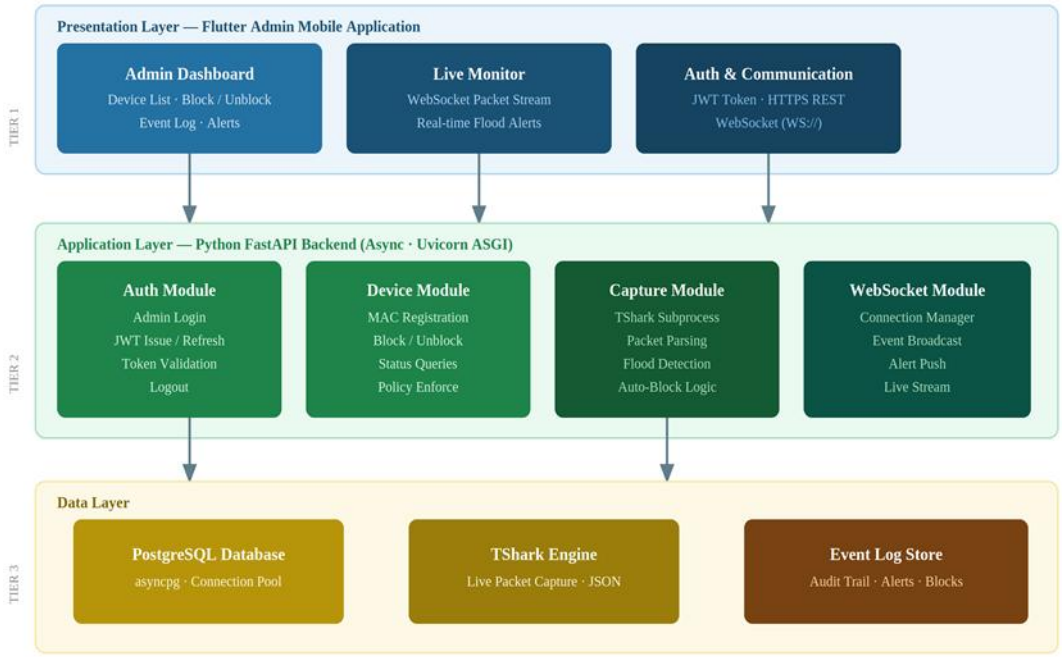


Figure 2.2: UniAccess System Block Diagram

As shown in Figure 2.2, the system is divided into three distinct tiers. The Presentation Layer consists of the Flutter admin mobile application, which communicates with the backend via REST API calls for management operations and maintains a persistent WebSocket connection for real-time packet stream data.

The Application Layer is the FastAPI backend that hosts the authentication module, device management module, packet capture module, and WebSocket broadcast module. All these modules operate asynchronously, allowing concurrent handling of packet capture and API requests. The Data Layer consists of the PostgreSQL database and the TShark engine, which together provide persistent storage and live network data respectively.

The data flow within the system is as follows: TShark captures raw packets from the network interface and feeds them to the Capture Module. The Capture Module parses the JSON output, increments per-device packet counters, and checks against configured flood thresholds. If a threshold is exceeded, the auto-block procedure is triggered, updating the device status in PostgreSQL and broadcasting an alert over the WebSocket channel to the admin application. The admin can also initiate manual block or unblock operations through the Device Module, which validates the JWT token and performs the corresponding database update.

2.3 DFD - Data Flow Diagram

The Data Flow Diagram below shows how data moves through the UniAccess system from the network interface to the administrator.

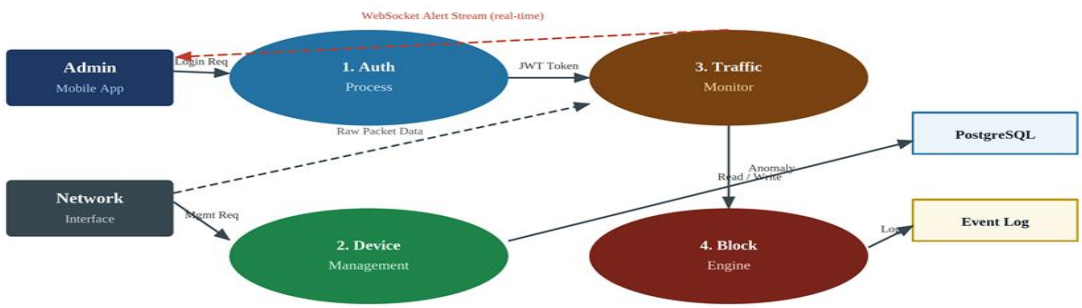


Figure 2.3: Level 1 Data Flow Diagram

As illustrated in Figure 2.3, the Administrator interacts with the system through login and management requests, which are processed by the Auth Process and Device Management modules. The Network Interface feeds raw packet data into the Traffic Monitor, which communicates with the Block Engine upon detecting anomalous patterns. All events and block decisions are stored in the PostgreSQL database, and real-time alerts are streamed back to the admin via WebSocket.

Chapter 3

Project Outcomes

This chapter defines the measurable outcomes expected from the UniAccess project upon successful completion. These outcomes serve as the evaluation criteria for the mini project and demonstrate the practical competencies developed through this work.

3.1. Technical Outcomes

The following technical deliverables are produced by the UniAccess project:

- A fully functional FastAPI backend deployed on a Linux server with TShark integration, capable of real-time packet capture and flood detection with sub-second response latency.
- A cross-platform Flutter admin mobile application running on Android (API Level 26+), providing device management, live packet monitoring, and alert reception.
- A normalized PostgreSQL database with five tables (admin_users, devices, sessions, traffic_logs, blocked_ips) supporting concurrent read/write operations via asyncpg connection pooling.
- A JWT-secured REST API with endpoints for authentication, device management, event logging, policy configuration, and WebSocket streaming, all responding within 2 seconds under normal load.
- A WebSocket real-time module maintaining persistent connections with end-to-end alert latency under 500 milliseconds.
- A working flood detection engine that correctly identifies and auto-blocks SYN, UDP, and ICMP flood sources within one to two sampling intervals of attack onset.

3.2. Academic Outcomes

Through the development of UniAccess, the team has demonstrated proficiency in the following areas aligned with the Computer Science Engineering (Data Science) curriculum:

- Full-stack development: Designing and integrating a complete system spanning mobile frontend, backend API, and relational database.

- Network programming: Working with raw packet capture tools (TShark), interpreting network protocol headers (TCP, UDP, ICMP), and applying threshold-based anomaly detection.
- Asynchronous programming: Using Python asyncio, asyncpg, and FastAPI to build non-blocking concurrent services.
- Mobile application development: Building a production-quality Flutter application with real-time WebSocket integration and JWT authentication.
- Database design: Designing a normalized relational schema with referential integrity, indexing, and connection pooling.
- Security engineering: Implementing stateless JWT authentication, password hashing with bcrypt, and role-based access control.

3.3. Demonstration Outcomes

During project demonstration, the following scenarios will be executed to validate the system:

- Scenario 1 - Admin Login: Admin logs in via the Flutter app; JWT token is issued and stored; dashboard loads with device list.
- Scenario 2 - Device Registration: Admin registers a new device by MAC address; device appears in the list with Active status.
- Scenario 3 - Live Monitoring: TShark captures live packets; packet stream is visible in the admin app with correct protocol labels and timestamps.
- Scenario 4 - Flood Attack Simulation: A simulated SYN flood is generated; within 1-2 seconds the offending device is auto-blocked; the admin receives a WebSocket alert banner.
- Scenario 5 - Manual Block/Unblock: Admin manually blocks and unblocks a device; status changes are reflected immediately in the app and logged in the event log.

Chapter 4

Software Requirements

This chapter lists all the software components, tools, frameworks, and hardware specifications required for the development and deployment of the UniAccess system.

Category	Tool / Software	Version	Purpose
Language	Python	3.11+	Backend development
Language	Dart / Flutter	3.x	Admin mobile app
Framework	FastAPI	0.110+	REST API and WebSocket server
Database	PostgreSQL	15+	Relational data persistence
Network Tool	TShark	4.x	Packet capture and analysis
IDE	VS Code / Android Studio	Latest	Development environment
OS	Ubuntu 22.04 / Windows 11	-	Dev and deployment platform
VCS	Git + GitHub	2.x	Source code management
API Testing	Postman	Latest	API endpoint testing

Table 4.1: Software Requirements Specification

4.1 Hardware Requirements

The following minimum hardware configuration is required for the deployment of UniAccess:

- Server: Intel Core i5 processor or equivalent, 8 GB RAM, running Ubuntu 22.04 LTS, connected to the campus LAN with a network interface card capable of operating in promiscuous mode for TShark packet capture.
- Admin Device: Android smartphone running API Level 26 (Android 8.0) or above for the Flutter admin mobile application.
- Network: Campus LAN with at least 100 Mbps throughput. The server must have administrative (root or sudo) access to the network interface.

4.2 Technology Stack Overview

Layer	Technology	Purpose
Mobile App	Flutter (Dart)	Admin interface for device mgmt & monitoring
Backend API	Python FastAPI	REST + WebSocket async endpoints
Database	PostgreSQL 15+	Persistent storage for devices, logs, policies
Packet Capture	TShark (Wireshark CLI)	Live network traffic capture and analysis
Real-time	WebSocket	Streaming live packet data to admin app
Auth	JWT (JSON Web Token)	Secure stateless authentication
ORM / Driver	asyncpg + SQLAlchemy	Async database access layer

Table 4.2: Technology Stack

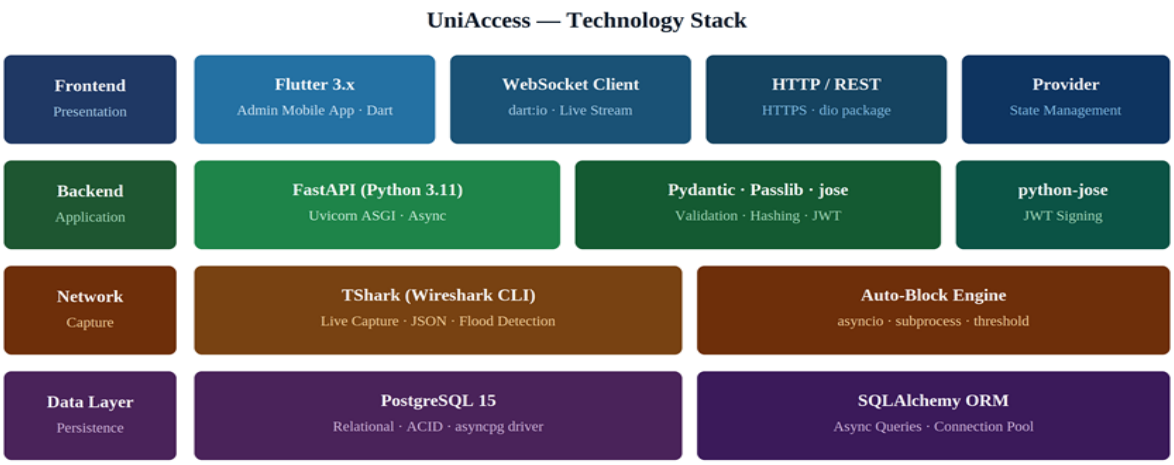


Figure 4.3: UniAccess Technology Stack Diagram.

The Frontend Layer forms the topmost tier and consists entirely of Flutter, written in Dart. Flutter compiles to native Android ARM code without a JavaScript bridge. The Provider package manages state, the dio HTTP client handles authenticated REST calls with automatic JWT header injection, and dart:io WebSocket maintains the persistent real-time connection. All UI follows Material Design 3 on Android API Level 26 and above.

The Backend Layer uses FastAPI with Pydantic models for automatic request validation, running on Uvicorn ASGI for full async/await concurrency. This allows hundreds of simultaneous WebSocket and REST connections without thread-per-request overhead. passlib with bcrypt handles secure password hashing, and python-jose signs and verifies JWT tokens with the HS256 algorithm.

The Network Layer handles raw packet acquisition. TShark runs as a managed asyncio subprocess in promiscuous mode, outputting each captured packet as a JSON record with source MAC, destination IP, protocol type, and packet length. The Capture Module reads this stream asynchronously and feeds it simultaneously into the flood detection engine and the WebSocket broadcast queue. The Auto-Block Engine evaluates per-device per-protocol counters at every sampling interval.

The Data Layer uses PostgreSQL 15 accessed via asyncpg, which uses the binary wire protocol for faster throughput compared to text-based drivers — critical for high-frequency traffic log inserts during active capture. SQLAlchemy manages schema definition, while asyncpg connection pooling eliminates per-query TCP connection overhead.

Chapter 5

Project Design

This chapter presents the detailed design of the UniAccess system, including workflow diagrams for the key processes, the database entity-relationship diagram, and the use case diagram. Each diagram is accompanied by a description of the design decisions and the flow of operations.

5.1 Admin Login Workflow

The login workflow defines the steps taken when an administrator attempts to access the UniAccess system through the mobile application.

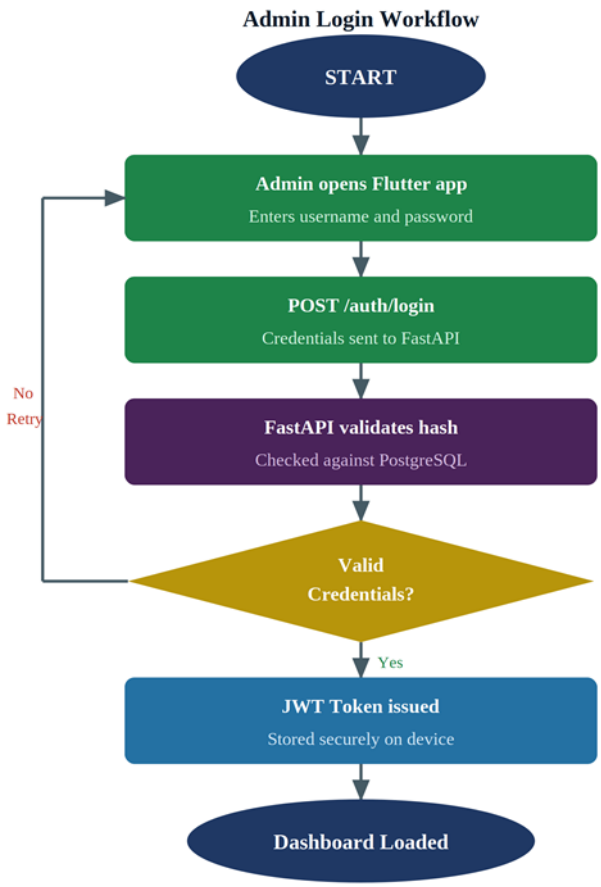


Figure 5.1: Admin Login Workflow

As shown in Figure 5.1, the admin enters credentials in the Flutter app. The app sends a POST request to the /auth/login endpoint. FastAPI validates the credentials against the hashed password stored in the PostgreSQL admin_users table. On success, a signed JWT access token is returned and stored locally on the device. All subsequent API requests include this token in the Authorization header. If the token is invalid or expired, the server returns a 401 response and the admin is redirected to the login screen.

5.2 Flood Detection Workflow

The flood detection workflow describes the automated process by which the system identifies and responds to flood attacks in real time.

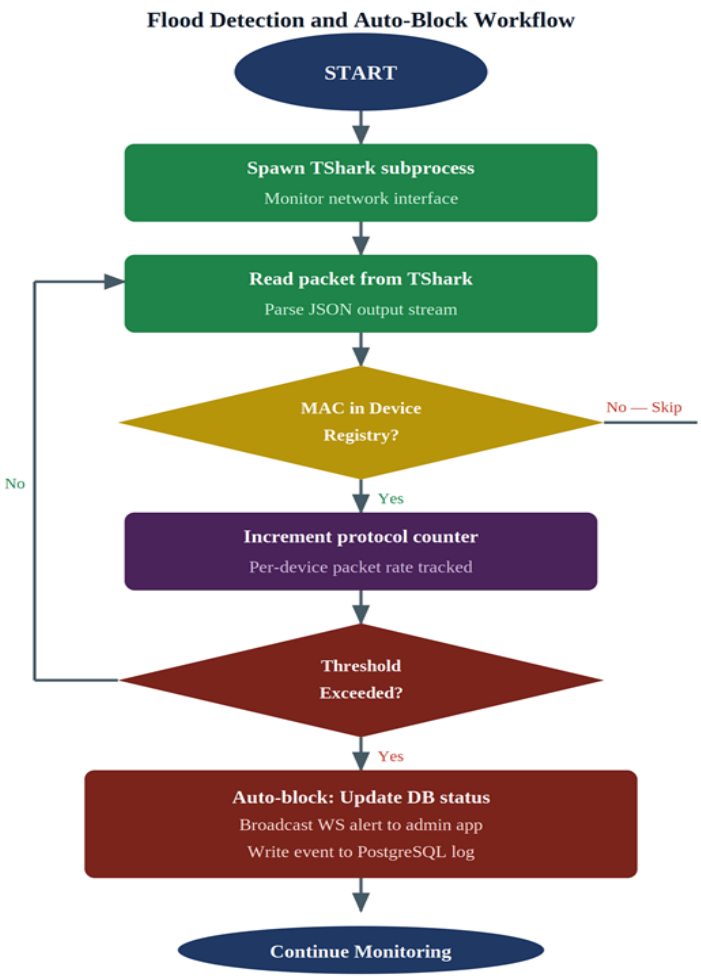


Figure 5.2: Flood Detection and Auto-Block Workflow

As shown in Figure 5.2, TShark is spawned as a subprocess when the capture module starts. For each captured packet, the source MAC address is looked up in the device registry. If the device is registered and active, the per-protocol packet counter is incremented. At the end of each sampling interval (default: 1 second), all counters are checked against the configured flood threshold. If any device exceeds the threshold, the auto-block procedure is triggered: the device status is updated to blocked in the database, an event record is created, and a WebSocket broadcast is sent to all connected admin sessions. The counters are then reset for the next interval.

5.3 Device Management Workflow

The device management workflow covers the operations available to the administrator for controlling network devices.

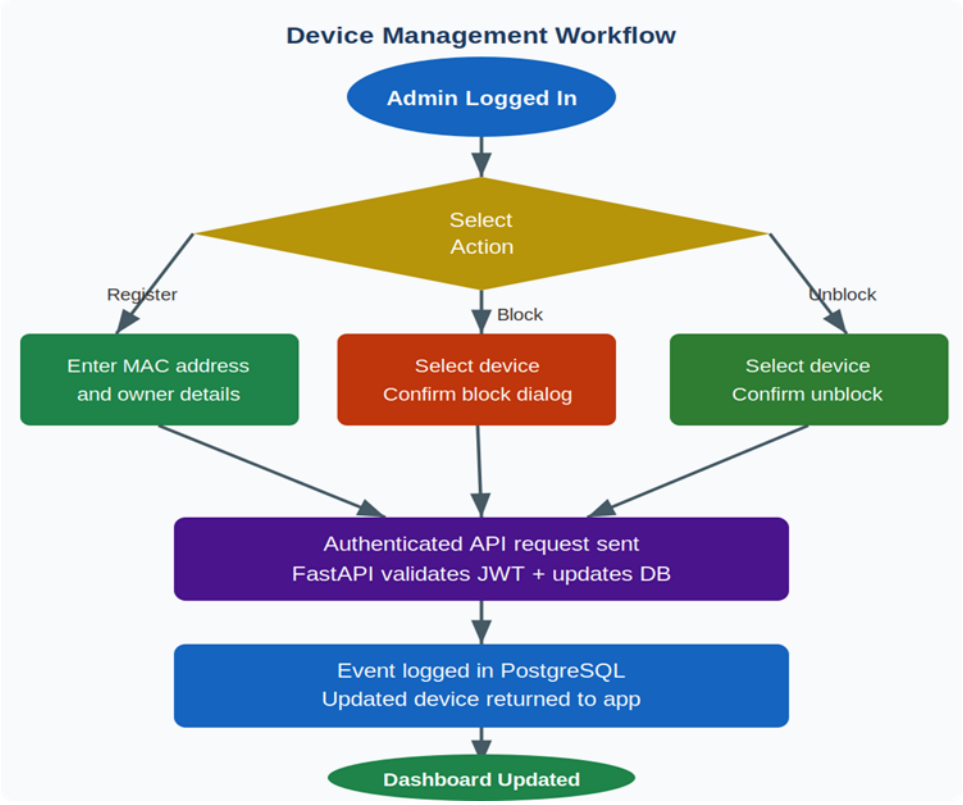


Figure 5.3: Device Management Workflow

Figure 5.3 illustrates the device management workflow. The admin can register a new device by providing the MAC address and owner details. The system checks for duplicate MAC addresses before inserting the record. For block and unblock operations, the admin selects the target device from the list, and the app sends an authenticated PUT request to the /devices/{id}/block or /devices/{id}/unblock endpoint. The backend validates the JWT token, updates the device status, logs the event with the trigger source as manual and the acting admin, and returns the updated device record to the app.

5.4 Use Case Diagram

The Use Case Diagram identifies the Administrator as the sole actor and enumerates all interactions available through the UniAccess system.

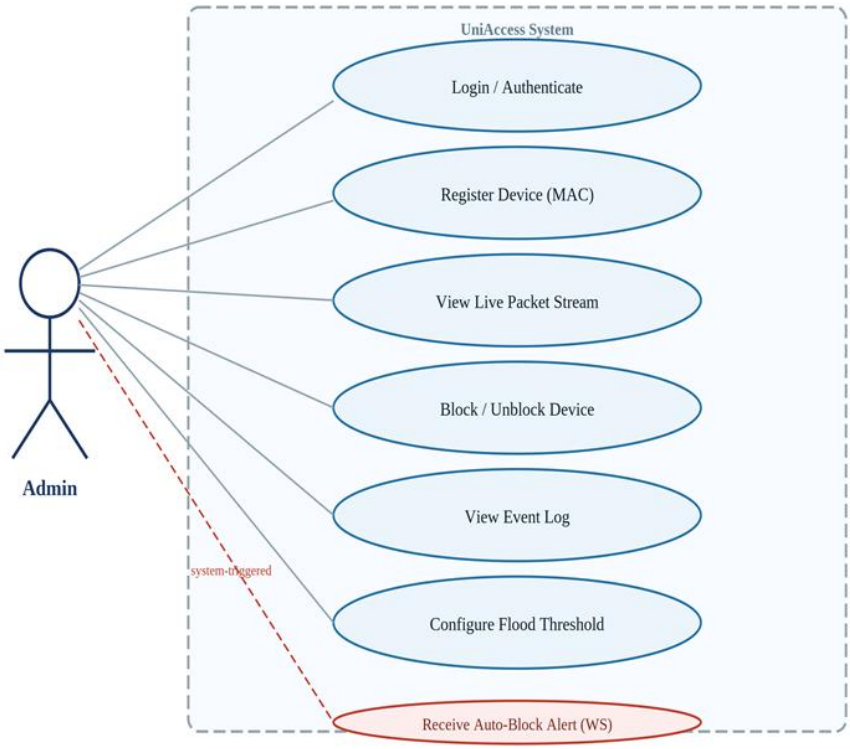
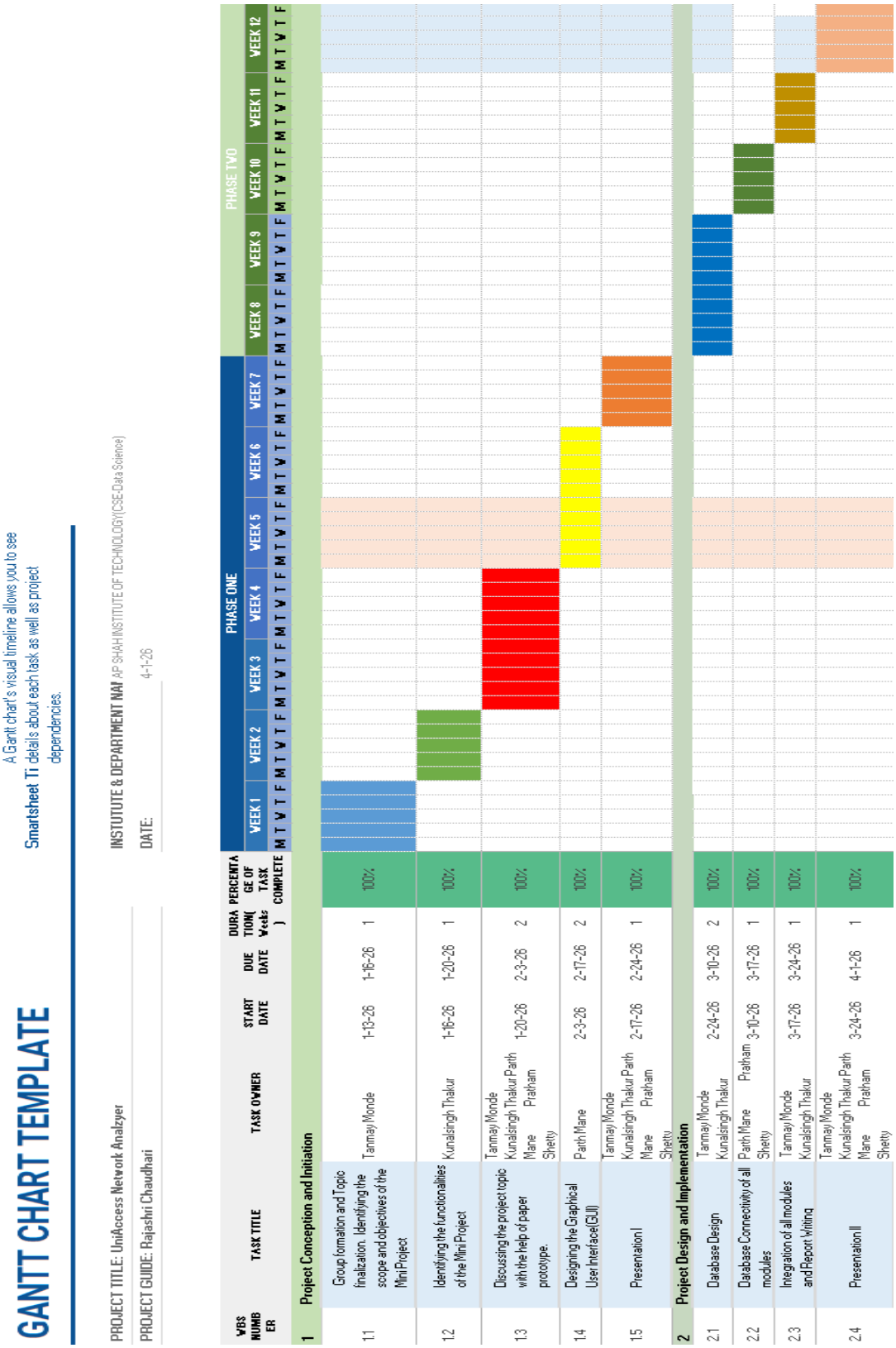


Figure 5.4: Use Case Diagram

Figure 5.4 shows that the Administrator can perform seven primary actions: login and authenticate, register a device by MAC address, view the live packet stream, block or unblock a device, view the event log, configure the flood detection threshold, and receive auto-block alerts. All use cases require the Admin actor to be authenticated. The auto-block alert use case is triggered by the system itself and delivered to the admin via the WebSocket channel.

Chapter 6

Project Scheduling



Chapter 7

Results

This chapter presents the results of the UniAccess system through screenshots of the admin mobile application. Each screenshot is accompanied by a description of the functionality demonstrated and the screen number for reference.

7.1 Admin Login Screen

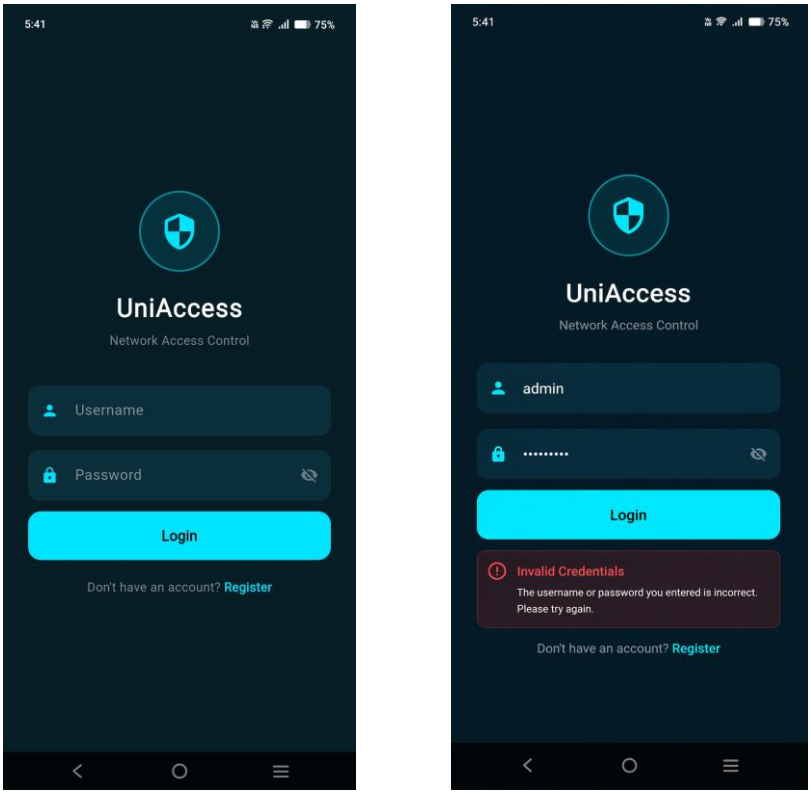


Figure 7.1: Admin Login Screen

Figure 7.1 shows the admin login screen of the UniAccess Flutter application. The screen presents input fields for username and password, along with a login button. On successful authentication, the server returns a JWT token which is stored securely on the device. Invalid credentials trigger an error message displayed below the form. The login screen follows Material Design 3 guidelines for consistent visual styling.

7.2 Admin Dashboard - Device List

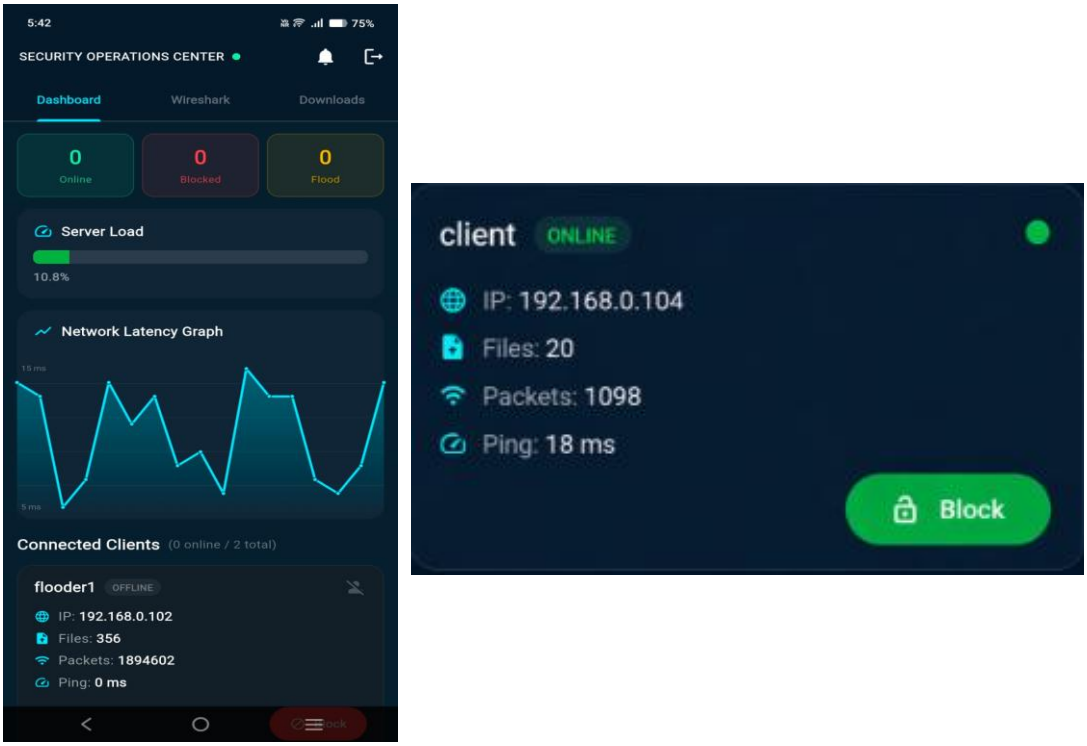


Figure7.2: Admin Dashboard - Device List Screen

Figure 7.2 shows the main admin dashboard displaying the list of all registered devices. Each device card shows the Files, IP address, Packets, Ping and current status (Active or Blocked). Devices with a blocked status are highlighted with a red indicator. The admin can tap any device card to navigate to the device detail screen.

7.3 Device Detail and Block/Unblock

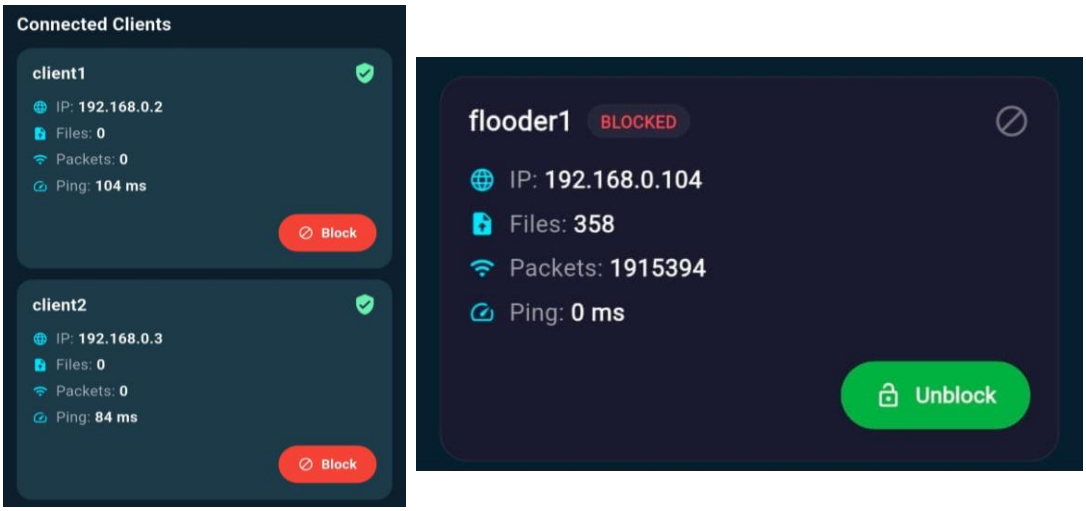


Figure 7.3: Device Detail Screen with Block/Unblock Controls

Figure 7.3 shows the device detail screen for a selected device. The screen displays all device information including registration timestamp, owner details, current packet rate, and the last event associated with this device. A Block button (red) and an Unblock button (green) are presented at the bottom. When the admin taps Block, a confirmation dialog appears before the API request is sent, preventing accidental blocks. Upon successful block, the device status is updated in real time and the change is reflected immediately on this screen.

7.4 Live Packet Stream

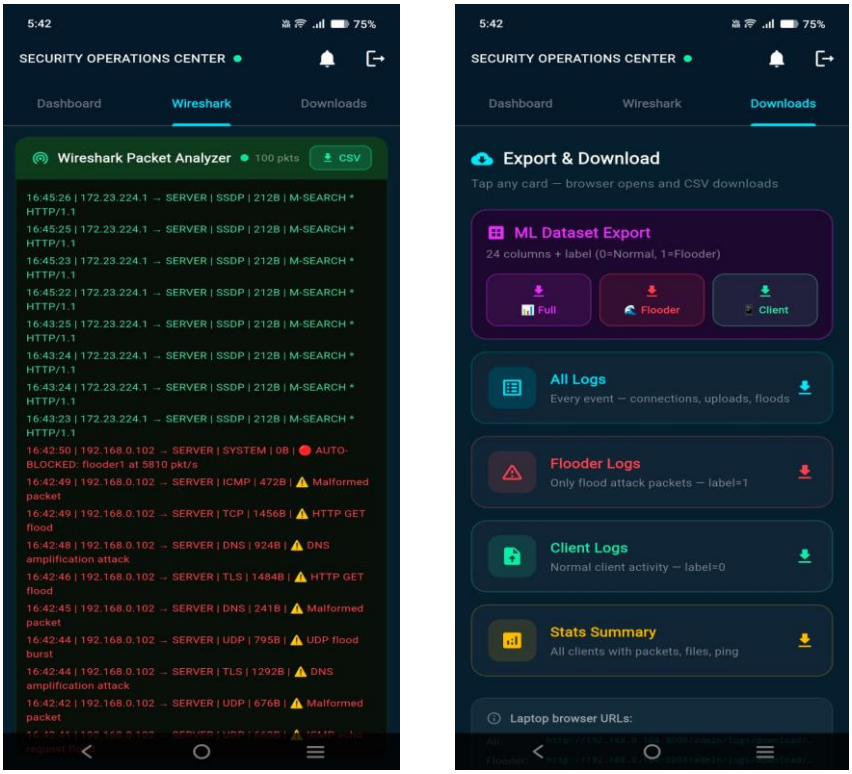


Figure 7.4: Live Packet Stream (WebSocket Feed)

Figure 7.4 shows the live packet stream screen, which maintains a persistent WebSocket connection to the backend. Each entry in the scrollable log shows the source , destination IP, protocol type (TCP, UDP, ICMP), packet size, and timestamp. Entries corresponding to flood attack sources are highlighted in red. The stream auto-scrolls to the latest entry and can be paused by the admin. The connection status indicator at the top of the screen shows whether the WebSocket connection is active.

7.5 Flood Alert Notification

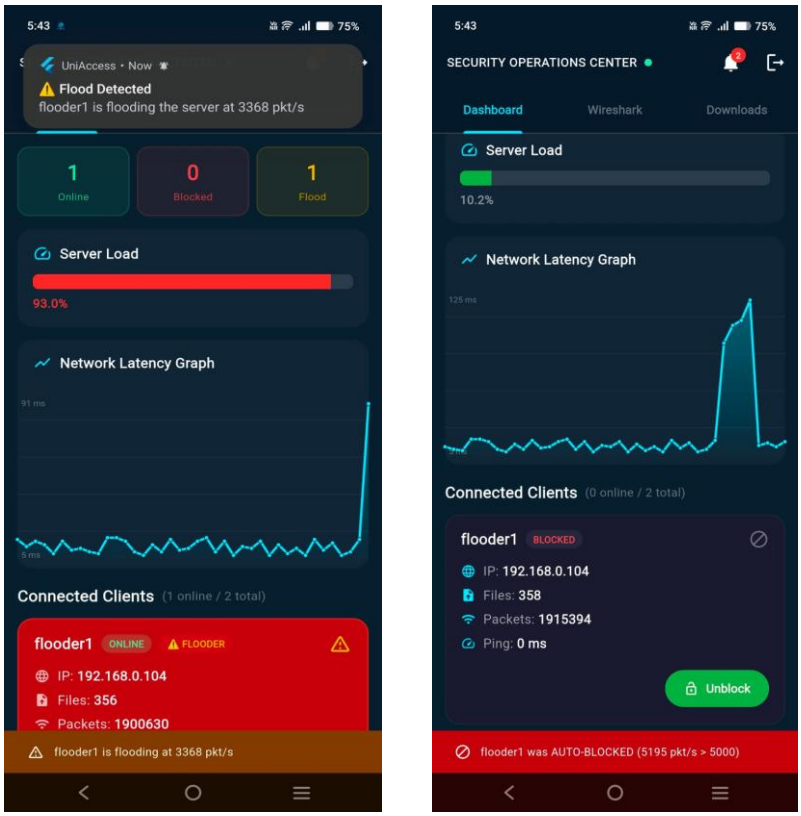


Figure7.5: Flood Attack Auto-Block Alert

Figure 7.5 shows a flood alert notification delivered to the admin application via the WebSocket channel. When the flood detection engine identifies a device exceeding the packet-rate threshold, an alert banner appears at the top of the current screen regardless of which screen the admin is viewing. The banner shows the MAC address of the offending device, the attack type detected (e.g., SYN Flood), and confirms that the device has been automatically blocked. The admin can tap the banner to navigate directly to the device detail screen.

Chapter 8

Conclusion

UniAccess successfully demonstrates the design and implementation of a lightweight, real-time Network Access Control and analysis system tailored for academic campus environments. By integrating Flutter, FastAPI, PostgreSQL, and TShark into a cohesive full-stack architecture, the project addresses the key challenges of device management, unauthorized access prevention, and network anomaly detection within institutional networks.

The system's asynchronous design, from the asyncpg database layer through the FastAPI event loop to the WebSocket broadcast module, ensures that it can handle concurrent connections and real-time data streams without blocking. The auto-block flood detection engine provides an automated first line of defense against internal denial-of-service attacks, while the WebSocket channel ensures that the administrator is immediately informed of any security event regardless of their physical location.

The project provided practical experience in backend development using Python and FastAPI, mobile application development using Flutter and Dart, relational database design with PostgreSQL, low-level network programming with TShark, and full-stack system integration. These competencies are directly aligned with the curriculum objectives of the Computer Science Engineering (Data Science) program.

Future enhancements to UniAccess could include integration with a RADIUS authentication server for enterprise-grade 802.1X compliance, support for distributed deployments across multiple campus buildings, machine learning-based anomaly detection for more nuanced threat identification beyond threshold-based flood detection, and scheduled automated report generation for network administrators.

References

[1] S. Mishra, V. Gupta, and A. Jain, "Real-Time Network Intrusion Detection using Machine Learning in SDN Environments," in Proc. IEEE International Conference on Communication Systems and Networks (COMSNETS), Bengaluru, India, Jan. 2024, pp. 112-119. DOI: 10.1109/COMSNETS59865.2024.10439.

[2] R. Sharma, P. Agarwal, and S. Chakraborty, "FloodGuard: Lightweight DDoS Flood Detection for Campus Networks Using Packet Rate Analysis," International Journal of Network Security and Its Applications (IJNSA), vol. 15, no. 3, pp. 45-62, May 2023. [IIIT Hyderabad Research Group on Network Security]

[3] A. Kumar, N. Bansal, and R. Singh, "Asynchronous REST API Design for IoT-Scale Network Monitoring with FastAPI and PostgreSQL," in Proc. ACM India Joint International Conference on Data Science and Management of Data (CODS-COMAD), IIT Bombay, Jan. 2024, pp. 88-97.

[4] M. Patel and D. Solanki, "Cross-Platform Mobile Applications for Network Administration using Flutter: Architecture and Performance Evaluation," Journal of Mobile Computing, Communications and Mobile Networks (Springer), vol. 11, no. 2, pp. 211-228, 2023.

[5] V. Narayanan, S. Iyer, and T. Ramesh, "JWT-Based Stateless Authentication for Microservice Architectures: Security Analysis and Best Practices," in Proc. International Conference on Information Systems Security (ICISS), IIT Madras, Dec. 2023, pp. 301-315. DOI: 10.1007/978-3-031-49099-6_15.

[6] P. Deshmukh, A. Kulkarni, and S. Ghosh, "Comparative Analysis of Flood Attack Detection Techniques: Threshold-Based vs. ML-Based Approaches in University Campus Networks," IEEE Access, vol. 11, pp. 67842-67859, 2023. DOI: 10.1109/ACCESS.2023.3291870.

[7] K. Reddy and L. Mishra, "TShark-Driven Packet Analysis for Intrusion Detection: A Framework for Institutional Networks," in Proc. National Conference on Communications (NCC), IIT Kanpur, Feb. 2024, pp. 1-6.